Introduction to R

Aravind Hebbali

2026-11-13

Contents

Preface 4
Structure of the book 4
Software information 5
Introduction to R 6
Overview 6
GUI 6
RStudio 6
Key points 6
Install R & RStudio 6
R 7
RStudio 7
Key points 7
Variables in R 7
Introduction 7
What is a variable? 7
Creating Variables 8
Using Variables 8
Components of a Variable 9
Naming Conventions 9
Key points 9
Data Types in R 9
Numeric 10
Integer 10
Character 11
Logical 12
Date 13
Key points 14
Vectors in R 14
Introduction 14
Numeric Vector 15
Integer Vector 15

Character Vector 16
Logical Vector 16
Naming Vector Elements 17
Method 1: Create vector and add names later 17
Method 2: Specify names while creating vector 17
Vector Coercion 17
Case 1: Different Data Types 18
Case 2: Numeric, Integer and Logical 18
Case 3: Integer and Logical 18
Vector Operations 18
Case 1: Vectors of same length 19
Case 2: Vectors of different length 19
Missing Data 20
Detect missing data 20
Omit missing data 20
Exclude missing data 20
Index Vectors 21
Out of range index 21
Negative Index 22
Subset Multiple Elements 22
Try it in your browser 23
Subset Named Vectors 24
Subset using logical values 24
Subset using logical expressions 25
Logical Operators 26
Practice Check 26
Key points 27
Dataframes in R 27
Introduction 28
Create dataframes 28
Basic Information 28
Select Columns 29
Single Column 29
Multiple Columns 29
Select Rows 29
Try it in your browser 30
Practice Check 30
Key points 31
Install & Update R Packages 31
Introduction 32
Install Packages 32
CRAN 32
Load a package 32

Update Packages ... 32
Key points ... 32
RStudio Projects & Data Import ... 32
Why avoid `setwd()` ... 33
Portable paths with `here` ... 33
Reading CSV files ... 33
Reading Excel files ... 33
Try it in your browser ... 34
Practice Check ... 34
Key points ... 35
Your First Visualization ... 35
Base R graphics ... 36
ggplot2 graphics ... 36
Try it in your browser ... 37
Practice Check ... 37
Key points ... 38
Your First Data Wrangling ... 38
The five verbs ... 39
Try it in your browser ... 39
Practice Check ... 40
Where to go next ... 41
Key points ... 41
Factors in R ... 41
Introduction ... 41
Create Factors ... 41
Order Levels ... 42
Specify Labels ... 42
Check Levels ... 42
Key points ... 42
Lists in R ... 43
Introduction ... 43
Creating Lists ... 43
Access List Elements ... 44
Coerce other objects ... 45
Coerce list to other objects ... 46
Practice Check ... 47
Key points ... 47
Getting Help in R ... 48
Introduction ... 48
Online Resources ... 48
R Bloggers ... 48
Stack Overflow ... 48
Posit Community ... 48

RStudio ... 49
Reddit ... 49
R Weekly ... 49
R User Groups ... 49
Internal ... 49
help.start ... 49
help ... 49
help.search ... 49
demo ... 50
example ... 50
Package Documentation ... 50
library ... 50
vignette ... 50
browseVignettes ... 50
RSiteSearch ... 50
Key points ... 51
About the Author ... 51
Package management ... 51
Introduction ... 51
Other package sources ... 51
GitHub ... 51
Bioconductor ... 52
Install Different Versions ... 52
Installed Packages ... 52
Library Paths ... 52
Install ... 53
Load ... 53
Update Packages ... 53
Remove Packages ... 53
R background and installation tools ... 53
Organizations ... 53
R Foundation ... 53
R Consortium ... 53
Installation ... 53
CRAN ... 53
Extensions ... 53
R GUI ... 54

Preface

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

Structure of the book

Chapter Introduction to R gives a quick overview of R and its development tools. Chapter Install R & RStudio guides you through installing R and RStudio. Chapter Variables in R introduces variables and assignment. Chapter Data Types in R introduces the types of values used in R. Chapter Vectors in R covers vectors, coercion, operations and indexing. Chapter Dataframes in R covers creating and selecting rows and columns in data frames. Chapter Install & Update R Packages covers installing, loading and updating packages. Chapter RStudio Projects & Data Import covers portable paths and reading CSV and Excel files. Chapter Your First Visualization introduces base R and ggplot2 graphics. Chapter Your First Data Wrangling introduces dplyr verbs and the native pipe. Chapter Factors in R covers categorical data, levels and labels. Chapter Lists in R covers storing and extracting elements of different types. Chapter Getting Help in R shows where and how to ask for help and find documentation. About the Author follows the chapters, and Package management collects reference material in an appendix.

Software information

The R session information below shows the setup used to compile this book:

```
sessionInfo()
```

```
R version 4.5.3 (2026-03-11)
Platform: x86_64-pc-linux-gnu
Running under: Ubuntu 24.04.5 LTS

Matrix products: default
BLAS:   /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblasp-r0.3.26.so;
LAPACK version 3.12.0

locale:
 [1] LC_CTYPE=C.UTF-8       LC_NUMERIC=C           LC_TIME=C.UTF-8
 [4] LC_COLLATE=C.UTF-8     LC_MONETARY=C.UTF-8    LC_MESSAGES=C.UTF-8
 [7] LC_PAPER=C.UTF-8       LC_NAME=C              LC_ADDRESS=C
[10] LC_TELEPHONE=C         LC_MEASUREMENT=C.UTF-8 LC_IDENTIFICATION=C

time zone: UTC
tzcode source: system (glibc)

attached base packages:
[1] stats     graphics  grDevices datasets  utils     methods   base

loaded via a namespace (and not attached):
 [1] compiler_4.5.3  fastmap_1.2.0   cli_3.6.6       htmltools_0.5.9
 [5] tools_4.5.3     otel_0.3.0      yaml_2.3.12     rmarkdown_2.32
 [9] knitr_1.52      jsonlite_2.0.0  xfun_0.62       digest_0.6.39
[13] rlang_1.3.0     renv_1.2.4      evaluate_1.0.5
```

We do not add prompts (> and +) to R source code in this book. We comment out text output with two hashes ## by default, as in the R session information above. This lets you copy and run the code, since R ignores the commented text output. Package names are in bold text (e.g.,

rmarkdown), and function names are followed by parentheses (e.g., `mean()`). The double-colon operator `::` means accessing an object from a package.

Introduction to R

Learning Objectives: After this chapter you will be able to:

- Describe what R is and why it is widely used for data science.
- Distinguish RStudio Desktop from Positron as an IDE for R.
- Locate the panes you will work in: script editor, console, and output.

Overview

R is a free and open source programming language widely used in academia, industry and government organizations for data science and machine learning. R is well suited for interactive data analysis, visualization and modeling. To view a list of organizations using R, click here.

Background on R organizations, package sources and the R GUI is in Package management.

GUI

RStudio

RStudio is a popular IDE (integrated development environment) built for the data science workflow. RStudio is developed by Posit (formerly RStudio, rebranded in 2022); Positron is a modern alternative IDE for R and Python.

Key points

- R is a free and open source language for data analysis, visualization and modeling.
- RStudio is an IDE that supports the data science workflow.
- Positron is another IDE for working with R and Python.

Next: Install R & RStudio

Course companion: Course 1, lesson 0

Install R & RStudio

Learning Objectives: After this chapter you will be able to:

- Install R on Windows, macOS, or Linux from a CRAN mirror.
- Verify an installation by running `R.version.string`.
- Install RStudio Desktop after installing R.
- Create an RStudio Project to keep the working directory, history, and settings together in a single `.Rproj` file.

R

Follow these steps:

- Download R for your operating system from a CRAN mirror (Windows, macOS, or Linux installers are listed there).
- Run the installer with default options.
- Verify the installation by opening R (or RStudio) and typing `R.version.string` in the Console.

RStudio

Follow these steps:

- Download RStudio Desktop from posit.co/download/rstudio-desktop.
- Install it after installing R (RStudio needs R to work).
- Open RStudio and create a new project via *File* > *New Project* (this creates an `.Rproj` file that keeps your working directory, history, and settings together).
- Verify your setup by running `R.version.string` in the R console.

Key points

- Install R from a CRAN mirror before installing RStudio Desktop.
- Type `R.version.string` in the R or RStudio Console to verify the installation.
- Create an RStudio Project to keep the working directory and settings together.

Next: Variables in R

Variables in R

Practise this interactively in Course 1 (Introduction to R), lesson 2

Learning Objectives: After this chapter you will be able to:

- Explain what a variable is and why programs use them.
- Create and reassign variables with `<-`.
- Use variables in arithmetic and in function calls.
- Name variables legally, including the rule that permits a leading `..`
- Choose `snake_case` over periods, which are legal but discouraged.

Introduction

In the previous chapter, we learned to install RStudio. In this chapter, we will learn about variables. You can skip this chapter if you have prior experience in another programming language.

What is a variable?

- variables are the fundamental elements of any programming language
- they represent values that are likely to change
- they reference memory locations that store data

Let us use a simple case study to understand variables. Suppose you are computing the area of a circle whose radius is 3. In R, you can do this straight away as shown below:

```
3.14 * 3 * 3
```

```
[1] 28.26
```

Every new analysis or computation would need the radius and the area typed again. Let us see how variables help us reuse values and computations.

Creating Variables

A variable consists of 3 components:

- variable name
- assignment operator
- variable value

We can store the radius by assigning its value to a variable. In this case, we create a variable called `radius` and assign it the value `3` using the assignment operator `<-`.

```
radius <- 3
radius
```

```
[1] 3
```

Now that we have learned to create variables, let us see how we can use them for other computations. For our case study, we will use the `radius` variable to compute the area of a circle.

Using Variables

We will use `radius` and R's built-in constant `pi` to compute the area of a circle and store it in another variable `area`. R already provides `pi`; there is no need to assign it a value.

```
# assign value 3 to variable radius
radius <- 3

# compute area of circle
area <- pi * radius^2

# call radius
radius
```

```
[1] 3
```

```
# call area
area
```

```
[1] 28.27433
```

Components of a Variable

Naming Conventions

- A name must begin with a letter or a dot (`.`); e.g. `.hidden <- 42` is valid in base R. Do not use numbers, dollar sign (`$`) or underscore (`_`) as the first character.
- The name can contain numbers or underscores. Do not use dash (`-`). Periods (`.`) are syntactically valid in R variable names, but are discouraged in modern style guides (tidyverse style prefers `snake_case`).
- Do not use the names of keywords and avoid using the names of built-in functions.
- Variables are case sensitive; `average` and `Average` would be different variables.
- Use descriptive names. Generally, variable names should be nouns.
- If the name contains more than one word, use underscores to separate the words.

Key points

- Use `<-` to assign a value to a variable and the variable name to reuse it.
- Changing a variable lets you repeat a calculation with a new value.
- R already provides the constant `pi`.
- Variable names are case sensitive; choose descriptive names such as `snake_case`.

Next: Data Types in R

Practise interactively: Course 1, lesson 2

Data Types in R

Practise this interactively in Course 1 (Introduction to R), lesson 4

Learning Objectives: After this chapter you will be able to:

- Create numeric, integer, character, and logical values.
- Check a value's type with `is.numeric()`, `is.integer()`, `is.character()`, and `is.logical()`.
- Convert between types with `as.integer()`, `as.character()`, and friends.
- Know where to find implicit coercion of mixed types in the next chapter.
- Create and format a date, and recognize `POSIXct` for date-times.

Every value in R has a type, and the type decides what you can do with it. This chapter walks through the five you will meet first: `numeric` (double), `integer`, `character`, `logical`, and `date/time`. Most of the time you will let R infer the type for you, and then use the `is.*()` and `as.*()` functions to check or change it.

Numeric

In R, we represent numbers with the data type `numeric`. We will first create a variable and assign it a value. Next we will learn how to check the type of the variable.

```
# create two variables
number1 <- 3.5
number2 <- 3

# check data type
class(number1)
```

```
[1] "numeric"
```

```
class(number2)
```

```
[1] "numeric"
```

```
# check if data type is numeric
is.numeric(number1)
```

```
[1] TRUE
```

```
is.numeric(number2)
```

```
[1] TRUE
```

In the example above, R also treats `integers` as `numeric/double`. We will learn to create integers in a while. We used two new functions in the example above:

- `class()`: returns the `class` or `type`
- `is.numeric()`: tests whether the variable is of type `numeric`

Integer

Unless specified otherwise, integers are treated as `numeric` or `double`. Let us create variables of the type `integer` and convert other data types to `integer`.

- create a variable `number1` and assign it the value `3`
- check the data type of `number1` using `class`
- create a second variable `number2` using `as.integer` and assign it the value `3`
- check the data type of `number2` using `class`
- finally use `is.integer` to check the data type of both `number1` and `number2`

```
# create a variable and assign it an integer value
number1 <- 3

# create another variable using as.integer
number2 <- as.integer(3)

# check the data type
class(number1)
```

```
[1] "numeric"
```

```
class(number2)
```

```
[1] "integer"
```

```
# use is.integer to check data type
is.integer(number1)
```

```
[1] FALSE
```

```
is.integer(number2)
```

```
[1] TRUE
```

Character

We represent letters, words and groups of words with the data type `character`. We enclose data of type `character` in single or double quotes. R treats any value enclosed in quotes as `character`. Let us create two variables to store a first and last name.

```
# first name
first_name <- "jovial"

# last name
last_name <- 'mann'

# check data type
class(first_name)
```

```
[1] "character"
```

```
class(last_name)
```

```
[1] "character"
```

```
# use is.character to check data type
is.character(first_name)
```

```
[1] TRUE
```

```
is.character(last_name)
```

```
[1] TRUE
```

You can coerce any data type to `character` using `as.character()`.

```
# create variable of different data types
age <- as.integer(30) # integer
score <- 9.8          # numeric/double
opt_course <- TRUE    # logical
today <- Sys.time()   # date time

as.character(age)
```

```
[1] "30"
```

```
as.character(score)
```

```
[1] "9.8"
```

```
as.character(opt_course)
```

```
[1] "TRUE"
```

```
as.character(today)
```

```
[1] "2026-11-13 17:45:31.294325"
```

Logical

Logical data types take only 2 values. Either `TRUE` or `FALSE` (plus `NA` for missing values, giving three-valued logic in R). We create such data types when we compare two objects in R using comparison or logical operators.

- create two variables `x` and `y`
- assign them the values `TRUE` and `FALSE` respectively
- use `is.logical` to check data type
- use `as.logical` to coerce other data types to `logical`

```
# create variables x and y
x <- TRUE
y <- FALSE

# check data type
class(x)
```

```
[1] "logical"
```

```
is.logical(y)
```

```
[1] TRUE
```

The outcome of comparison operators is always `logical`. In the example below, we compare two numbers to see the outcome.

```
# create two numeric variables
x <- 3
y <- 4

# compare x and y
x > y
```

```
[1] FALSE
```

```
x < y
```

```
[1] TRUE
```

```
# store the result
z <- x > y
class(z)
```

```
[1] "logical"
```

`TRUE` is represented by all numbers except `0`. `FALSE` is represented only by `0`.

```
# TRUE and FALSE are represented by 1 and 0
as.logical(1)
```

```
[1] TRUE
```

```
as.logical(0)
```

```
[1] FALSE
```

```
# using numbers
as.numeric(TRUE)
```

```
[1] 1
```

```
as.numeric(FALSE)
```

```
[1] 0
```

```
# using different numbers
as.logical(c(-2, -1.5, -1, 0, 1, 2))
```

```
[1]  TRUE  TRUE  TRUE FALSE  TRUE  TRUE
```

Use `as.logical()` to coerce other data types to `logical`.

```
# create variable of different data types
age <- as.integer(30) # integer
score <- 9.8          # numeric/double
opt_course <- TRUE    # logical
today <- Sys.time()   # date time

as.logical(age)
```

```
[1] TRUE
```

```
as.logical(score)
```

```
[1] TRUE
```

```
as.logical(opt_course)
```

```
[1] TRUE
```

Date

R has dedicated types for dates and date-times. Use `Date` for calendar dates and `POSIXct` for date-times:

```
# today's date
today_date <- Sys.Date()
today_date
```

```
[1] "2026-11-13"
```

```
class(today_date)
```

```
[1] "Date"
```

```
# create a date from a string
launch_date <- as.Date("2026-03-01")
launch_date
```

```
[1] "2026-03-01"
```

```
# pretty-print a date
format(launch_date, "%B %d, %Y")
```

```
[1] "March 01, 2026"
```

For date-times, use `as.POSIXct()`. For arithmetic with dates, such as finding differences, adding months or handling time zones, use the lubridate package.

Key points

- Numeric, integer, character and logical values support different operations.
- Use `class()` and the `is.*()` functions to inspect a value.
- Use `as.*()` functions to convert values between types.
- Use `Date` for calendar dates and `POSIXct` for date-times.

Next: Vectors in R

Practise interactively: Course 1, lesson 4

Vectors in R

Practise this interactively in Course 1 (Introduction to R), lesson 3

Learning Objectives: After this chapter you will be able to:

- Create numeric, integer, character, and logical vectors.
- Name vector elements, and select elements by name.
- Apply arithmetic across a whole vector at once.
- Explain the coercion order `logical` → `integer` → `double` → `character`.
- Subset by position, including negative positions, and by comparisons.
- Find missing values with `is.na()` and exclude them with `na.rm = TRUE`.

Introduction

Vector is the most basic data structure in R. It is a sequence of elements of the same data type. If the elements are of different data types, R coerces them to a common type that can accommodate all of them. We generally create vectors using `c()`, the concatenate function. Depending on the data type, we can also use other methods.

Numeric Vector

We will create a numeric vector using `c()`, but you can use any function that creates a sequence of numbers. We will then use `is.vector()` to check if it is a vector and `class` to check the data type.

```
# create a numeric vector
num_vect <- c(1, 2, 3)

# display the vector
num_vect
```

```
[1] 1 2 3
```

```
# check if it is a vector
is.vector(num_vect)
```

```
[1] TRUE
```

```
# check data type
class(num_vect)
```

```
[1] "numeric"
```

Let us look at other ways to create a sequence of numbers. We leave it as an exercise to the reader to understand the functions using `help`.

```
# using colon
vect1 <- 1:10
vect1
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```
# using rep
vect2 <- rep(1, 5)
vect2
```

```
[1] 1 1 1 1 1
```

```
# using seq
vect3 <- seq(10)
vect3
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

Integer Vector

Creating an integer vector is similar to creating a numeric vector, except that we instruct R to treat the data as `integer` and not `numeric` or `double`. We use the same methods as before. To specify that the data is of type `integer`, we suffix the number with `L`:

```
# integer vector
int_vect <- c(1L, 2L, 3L)
int_vect
```

```
[1] 1 2 3
```

```
# check data type
class(int_vect)
```

```
[1] "integer"
```

```
# using colon
vect1 <- 1L:10L
vect1
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```
# using rep
vect2 <- rep(1L, 5)
vect2
```

```
[1] 1 1 1 1 1
```

```
# using seq
vect3 <- seq(10L)
vect3
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

Character Vector

A character vector may contain a single character, a word or a group of words. We enclose the elements in single or double quotes:

```
# character vector
greetings <- c("hello", "good morning")
greetings
```

```
[1] "hello"        "good morning"
```

```
# check data type
class(greetings)
```

```
[1] "character"
```

Logical Vector

A logical vector contains `TRUE`, `FALSE` or both:

```
# logical vector
vect_logic <- c(TRUE, FALSE, TRUE, TRUE, FALSE)
vect_logic
```

```
[1]  TRUE FALSE  TRUE  TRUE FALSE
```

```
# check data type
class(vect_logic)
```

```
[1] "logical"
```

In fact, you can create an `integer` vector and coerce it to type `logical`.

```r
# integer vector
int_vect <- rep(0L:1L, 3)
int_vect
```

```
[1] 0 1 0 1 0 1
```

```r
# coerce to logical vector
log_vect <- as.logical(int_vect)
log_vect
```

```
[1] FALSE  TRUE FALSE  TRUE FALSE  TRUE
```

```r
# check data type
class(log_vect)
```

```
[1] "logical"
```

Naming Vector Elements

We can name the elements of a vector. The advantage is that we can later use these names to access the elements. Use `names()` to specify the names of a vector. We can specify the names while creating the vector or add them later.

Method 1: Create vector and add names later

```r
# create vector and add names later
vect1 <- c(1, 2, 3)

# name the elements of the vector
names(vect1) <- c("One", "Two", "Three")

# call vect1
vect1
##   One   Two Three
##     1     2     3
```

Method 2: Specify names while creating vector

```r
# specify names while creating vector
vect2 <- c(John = 1, Jack = 2, Jill = 3, Jovial = 4)

# call vect2
vect2
##   John   Jack   Jill Jovial
##      1      2      3      4
```

Vector Coercion

Vectors are homogeneous: all the elements of a vector must be of the same type. If we create a vector by combining different data types, R coerces the elements to the most flexible type.

`character` is the most flexible data type while `logical` is the least flexible.

Case 1: Different Data Types

```
# vector of different data types
vect1 <- c(1, 1L, 'one', TRUE)

# call vect1
vect1
## [1] "1"    "1"    "one"  "TRUE"

# check data type
class(vect1)
## [1] "character"
```

Case 2: Numeric, Integer and Logical

```
# vector of different data types
vect1 <- c(1, 1L, TRUE)

# call vect1
vect1
## [1] 1 1 1

# check data type
class(vect1)
## [1] "numeric"
```

Case 3: Integer and Logical

```
# vector of different data types
vect1 <- c(1L, TRUE)

# call vect1
vect1
## [1] 1 1

# check data type
class(vect1)
## [1] "integer"
```

To summarize, coercion takes place in this order:

`logical` → `integer` → `double/numeric` → `character`

R coerces toward the **most flexible** type. If a vector contains a `character` element, everything becomes `character`. Without `character`, the elements become `numeric`; without `numeric`, they become `integer`.

Vector Operations

Let us look at simple operations we can perform on vectors in R. The nature of the operations depends upon the type of data. In the example below, we add, subtract, multiply and divide two vectors:

Case 1: Vectors of same length

```
# create two vectors
vect1 <- c(1, 3, 8, 4)
vect2 <- c(2, 7, 1, 9)

# addition
vect1 + vect2
## [1]  3 10  9 13

# subtraction
vect1 - vect2
## [1] -1 -4  7 -5

# multiplication
vect1 * vect2
## [1]  2 21  8 36

# division
vect1 / vect2
## [1] 0.5000000 0.4285714 8.0000000 0.4444444
```

Case 2: Vectors of different length

In the previous case, the length, i.e. the number of elements in the vectors, was the same. What happens if the lengths are unequal? In such cases, R recycles the shorter vector to match the length of the longer one. In the example below, we repeat the four operations on two vectors of different lengths:

```
# create two vectors
vect1 <- c(2, 7)
vect2 <- c(1, 8, 5, 2)

# addition
vect1 + vect2
## [1]  3 15  7  9

# subtraction
vect1 - vect2
## [1]  1 -1 -3  5

# multiplication
vect1 * vect2
## [1]  2 56 10 14

# division
vect1 / vect2
## [1] 2.000 0.875 0.400 3.500
```

Missing Data

Missing data is a reality. No matter how careful you are in collecting data for your analysis, you may end up with some missing values. In R, missing values are represented by `NA`. In this section, we will focus on the following:

- test for missing data
- remove missing data
- exclude missing data from analysis

Detect missing data

We first create a vector with missing values. Then we use `is.na()` to test whether the data contains missing values. `is.na()` returns a logical vector equal to the length of the vector being tested. `complete.cases()` is another function that detects missing values. In the example below, we use both functions on the same vector:

```
# vector with missing values
vect1 <- c(1, 3, NA, 5, 2)

# use is.na
is.na(vect1)
## [1] FALSE FALSE  TRUE FALSE FALSE

# use complete.cases
complete.cases(vect1)
## [1]  TRUE  TRUE FALSE  TRUE  TRUE
```

Omit missing data

Most summary functions (`mean()`, `sum()`, `max()`) return `NA` when any value is missing. To avoid this, we can remove the missing data before any computation. `na.omit()` removes all missing values from the data. Let us look at an example:

```
# vector with missing values
vect1 <- c(1, 3, NA, 5, 2)

# call vect1
vect1
## [1]  1  3 NA  5  2

# omit missing values
na.omit(vect1)
## [1] 1 3 5 2
## attr(,"na.action")
## [1] 3
## attr(,"class")
## [1] "omit"
```

Exclude missing data

Whenever you want to exclude missing values from a computation, use `na.rm` and set it to `TRUE`:

```r
# vector with missing values
vect1 <- c(1, 3, NA, 5, 2)

# compute mean
mean(vect1)
## [1] NA

# compute mean by excluding missing value
mean(vect1, na.rm = TRUE)
## [1] 2.75
```

> **Common Mistake:** `mean()` on a vector containing `NA` returns `NA`, not the mean of the values that are present. Add `na.rm = TRUE` to drop missing values first, and check whether dropping them is defensible for your data.

Index Vectors

One of the most important steps in data analysis is selecting a subset of data from a bigger data set. Indexing retrieves values individually or a set of values that meet a specific criterion. In this chapter, we look at various ways of indexing vectors.

`[]` is the index operator in R. We can use various expressions within `[]` to subset data. In R, **positions begin at 1 and not 0**. Let us look at the values in different positions:

```r
# fixed seed so this chapter re-renders deterministically
set.seed(42)

# random sample of 10 values
vect1 <- sample(10)
vect1
```

```
 [1]  1  5 10  8  2  4  6  9  7  3
```

```r
# return third element
vect1[3]
```

```
[1] 10
```

```r
# return seventh element
vect1[7]
```

```
[1] 6
```

Out of range index

```r
# random sample of 10 values
vect1 <- sample(10)
vect1
```

```
 [1]  8  7  4  1  5 10  2  6  9  3
```

```r
# return value at index 0
vect1[0]
```

```
integer(0)
```

```
# length of the vector
length(vect1)
```

```
[1] 10
```

```
# out of range index
vect1[11]
```

```
[1] NA
```

In the first case, we used the position 0, and in the second case the position 11, which is greater than the length of the vector. R returns an empty vector in the first case and `NA` in the second case.

Negative Index

A negative position drops the value at that position. Unlike other languages, R does not count backwards from the end of the vector. Let us look at an example to understand how a negative position works in R:

> **Common Mistake:** `vect1[-3]` does not mean "the third element from the end". A negative index **deletes** that position. Use `tail(vect1, 3)` when you actually want the last three elements.

```
# random sample of 10 values
vect1 <- sample(10)
vect1
```

```
 [1]  9 10  3  4  5  6  1  2  8  7
```

```
# drop third element
vect1[-3]
```

```
[1]  9 10  4  5  6  1  2  8  7
```

```
# drop seventh element
vect1[-7]
```

```
[1]  9 10  3  4  5  6  2  8  7
```

Subset Multiple Elements

If we do not specify anything within `[]`, R returns all the elements in the vector. We can specify the positions using any expression that generates a sequence of integers. Let us look at a few examples:

```
# random sample of 10 values
vect1 <- sample(10)
vect1
```

```
 [1]  3  1  2  6 10  8  4  5  7  9
```

```
# return all elements
vect1[]
```

```
 [1]  3  1  2  6 10  8  4  5  7  9
```

```
# return first 5 values
vect1[1:5]
```

```
[1]  3  1  2  6 10
```

```
# return all values from the 5th position
end <- length(vect1)
vect1[5:end]
```

```
[1] 10  8  4  5  7  9
```

If we use the colon to generate the positions, we have to specify both the starting and the ending position. Otherwise, R will return an error.

What if we want elements that are not in a sequence, as we saw in the last example? In such cases, we create a vector using `c()` and use it to extract values from the original vector:

```
# random sample of 10 values
vect1 <- sample(10)
vect1
```

```
 [1]  5  4  2  8  3  1 10  7  6  9
```

```
# extract 2nd, 5th and 7th element
select <- c(2, 5, 7)
vect1[select]
```

```
[1]  4  3 10
```

```
# extract elements in position 1 to 4, 6 and 9
select <- c(1:4, 6, 9)
vect1[select]
```

```
[1] 5 4 2 8 1 6
```

Try it in your browser

Edit the code below and press **Run**. It executes entirely in your browser via WebR — no R installation needed. The runtime downloads once, on this page only.

First run downloads the R runtime and any required packages (about 20 s); later runs reuse them.

```
{webr-r}
vect <- c(12, 7, 25, 3, 18, 9)

# single element, a range, and specific positions
vect[2]
vect[2:4]
vect[c(1, 3, 6)]
```

```
# negative indices drop elements
vect[-c(1, 2)]

# logical subsetting
vect[vect > 10]
```

Subset Named Vectors

We can subset vectors using the names of the elements. **When we use the names of the elements for subsetting, the names must be enclosed in single or double quotes**; otherwise, R will return an error. Let us look at a few examples:

```
vect1 <- c(score1 = 8, score2 = 6, score3 = 9)
vect1
```

```
score1 score2 score3
     8      6      9
```

```
# extract score2
vect1['score2']
```

```
score2
     6
```

```
# extract score1 and score3
vect1[c('score1', 'score3')]
```

```
score1 score3
     8      9
```

Subset using logical values

We can use logical values to subset vectors. They are not very flexible but can be used for simple indexing. In all the examples below, R recycles the logical vectors to match the length of the vector from which we subset data:

```
# random sample of 10 values
vect1 <- sample(10)
vect1
```

```
 [1]  6  2  4  3 10  5  9  7  8  1
```

```
# returns all values
vect1[TRUE]
```

```
 [1]  6  2  4  3 10  5  9  7  8  1
```

```
# empty vector
vect1[FALSE]
```

```
integer(0)
```

```
# values in odd positions
vect1[c(TRUE, FALSE)]
```

```
[1]  6  4 10  9  8
```

```
# values in even positions
vect1[c(FALSE, TRUE)]
```

```
[1] 2 3 5 7 1
```

Subset using logical expressions

We can use logical expressions to extract elements that meet specific criteria. This method is the most flexible and useful, as we can combine multiple conditions using relational and logical operators. Before we use logical expressions, let us spend some time understanding comparison and logical operators; we will use them extensively hereafter.

Comparison Operators

A comparison operator always returns a logical value, `TRUE` or `FALSE`. Let us see how we can use comparison operators to subset data:

```
# random sample of 10 values
vect1 <- sample(10)
vect1
```

```
 [1] 10  8  5  1  7  4  6  2  9  3
```

```
# return elements greater than 5
vect1 > 5
```

```
 [1]  TRUE  TRUE FALSE FALSE  TRUE FALSE  TRUE FALSE  TRUE FALSE
```

```
vect1[vect1 > 5]
```

```
[1] 10  8  7  6  9
```

```
# return elements greater than or equal to 5
vect1 >= 5
```

```
 [1]  TRUE  TRUE  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE FALSE
```

```
vect1[vect1 >= 5]
```

```
[1] 10  8  5  7  6  9
```

```
# return elements lesser than 5
vect1 < 5
```

```
 [1] FALSE FALSE FALSE  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE
```

```
vect1[vect1 < 5]
```

```
[1] 1 4 2 3
```

```
# return elements lesser than or equal to 5
vect1 <= 5
```

```
 [1] FALSE FALSE  TRUE  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE
```

```
vect1[vect1 <= 5]
```

```
[1] 5 1 4 2 3
```

```
# return elements equal to 5
vect1 == 5
```

```
 [1] FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
```

```
vect1[vect1 == 5]
```

```
[1] 5
```

```
# return elements not equal to 5
vect1 != 5
```

```
 [1]  TRUE  TRUE FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
```

```
vect1[vect1 != 5]
```

```
[1] 10  8  1  7  4  6  2  9  3
```

Logical Operators

Let us combine comparison and logical operators to create expressions and use them to subset vectors:

```
# random sample of 10 values
vect1 <- sample(10)
vect1
```

```
 [1]  4  9  3  8  5  6 10  2  1  7
```

```
# return all elements less than 8 or divisible by 3
vect1[(vect1 < 8 | (vect1 %% 3 == 0))]
```

```
[1] 4 9 3 5 6 2 1 7
```

```
# return all elements less than 7 or divisible by 2
vect1[(vect1 < 7 | (vect1 %% 2 == 0))]
```

```
[1]  4  3  8  5  6 10  2  1
```

Practice Check

Knowledge Check: *Index positions start at 1* What does `vect1[3]` return, and what does `vect1[0]` return?

Click to reveal answer **Answer:** `vect1[3]` returns the **third** element. `vect1[0]` returns an empty vector.

Explanation: R counts positions starting at 1, so the first element is `vect1[1]`. Position 0 does not exist, so you get nothing back rather than the first element the way a 0-based language would give you. **Knowledge Check:** *Out of range indices* `vect1` has 10 elements. What does `vect1[11]` return?

Click to reveal answer **Answer:** `NA`.

Explanation: Index 0 returns an empty vector, but an index that is past the end of the vector returns `NA` instead. An empty result means "you asked for a position that cannot exist"; `NA` means

"that element is missing". **Knowledge Check:** *Negative indices delete* What does `vect1[-3]` do — return the third element from the end, or something else?

Click to reveal answer **Answer:** It **drops** the third element and returns everything else.

Explanation: Unlike Python and most other languages, R does not count backwards from the end for you. A negative index removes that position, so `vect1[-3]` is "all elements except the third". Use `tail(vect1, 3)` if you actually want the last three. **Knowledge Check:** *Coercion to the most flexible type* What is the data type of `c(1, 1L, 'one', TRUE)`?

Click to reveal answer **Answer:** `character`, giving `c("1", "1", "one", "TRUE")`.

Explanation: Coercion runs `logical` → `integer` → `double/numeric` → `character`, always toward the most flexible type. The single `'one'` forces everything to `character`. **Knowledge Check:** *Coercion without a character element* What is the data type of `c(1, 1L, TRUE)`?

Click to reveal answer **Answer:** `numeric` (double), giving `c(1, 1, 1)`.

Explanation: With no `character` present, the most flexible type available is `numeric`, so the integer and the logical are promoted to double. You only fall back to `integer` when the data contains neither `character` nor `numeric`, as in `c(1L, TRUE)`. **Knowledge Check:** *Missing values in a computation* What does `mean(c(1, 3, NA, 5, 2))` return?

Click to reveal answer **Answer:** `NA`. Use `mean(c(1, 3, NA, 5, 2), na.rm = TRUE)` to get `2.75`.

Explanation: `NA` means "unknown", not "zero", so one unknown value makes the whole mean unknown. `na.rm = TRUE` drops the missing values first, leaving 1, 3, 5, 2 — whose mean is 11 / 4 = 2.75. Decide deliberately whether dropping missing values is appropriate for your data.

Key points

- A vector holds values of one type; mixing types can coerce its elements.
- Name elements to select them by name as well as by position.
- Positive indices select elements, negative indices exclude them, and logical indices keep matches.
- Use `is.na()` to find missing values and `na.rm = TRUE` to exclude them from summaries.

Next: Dataframes in R

Practise interactively: Course 1, lesson 3

Dataframes in R

Practise this interactively in Course 1 (Introduction to R), lesson 7

Learning Objectives: After this chapter you will be able to:

- Create a data frame from equal-length vectors.

- Inspect one with `str()`, `dim()`, `names()`, and `nrow()`/`ncol()`.
- Select columns with `[`, `[[`, and `$`.
- Select rows by position and by condition.
- Subset rows and columns independently.
- Explain why `stringsAsFactors` defaults to `FALSE` in R 4.0.0 and later.

Introduction

Data frames are lists of equal-length vectors, so they can hold columns of different types.

Create dataframes

Use `data.frame` to create data frames. The function syntax is:

```
args(data.frame)
```

```
function (..., row.names = NULL, check.rows = FALSE, check.names = TRUE,
    fix.empty.names = TRUE, stringsAsFactors = FALSE)
NULL
```

Data frames are lists with elements of equal length, so they can hold different data types. Let us create a data frame:

```
name <- c('John', 'Jack', 'Jill')
age <- c(29, 25, 27)
graduate <- c(TRUE, TRUE, FALSE)
students <- data.frame(name, age, graduate)
students
```

```
  name age graduate
1 John  29     TRUE
2 Jack  25     TRUE
3 Jill  27    FALSE
```

Basic Information

```
class(students)
## [1] "data.frame"
names(students)
## [1] "name"     "age"      "graduate"
colnames(students)
## [1] "name"     "age"      "graduate"
str(students)
## 'data.frame':    3 obs. of  3 variables:
##  $ name    : chr  "John" "Jack" "Jill"
##  $ age     : num  29 25 27
##  $ graduate: logi  TRUE TRUE FALSE
dim(students)
## [1] 3 3
nrow(students)
## [1] 3
```

```
ncol(students)
## [1] 3
```

Select Columns

Single Column

- []
- [[]]
- $

```
# using [
students[1]
##   name
## 1 John
## 2 Jack
## 3 Jill

# using [[
students[[1]]
## [1] "John" "Jack" "Jill"

# using $
students$name
## [1] "John" "Jack" "Jill"
```

Multiple Columns

```
students[, 1:3]
##   name age graduate
## 1 John  29     TRUE
## 2 Jack  25     TRUE
## 3 Jill  27    FALSE

students[, c(1, 3)]
##   name graduate
## 1 John     TRUE
## 2 Jack     TRUE
## 3 Jill    FALSE
```

Select Rows

```
# single row
students[1, ]
##   name age graduate
## 1 John  29     TRUE

# multiple row
students[c(1, 3), ]
##   name age graduate
## 1 John  29     TRUE
## 3 Jill  27    FALSE
```

Try it in your browser

Edit the code below and press **Run**. It executes entirely in your browser via WebR — no R installation needed. The runtime downloads once, on this page only.

First run downloads the R runtime and any required packages (about 20 s); later runs reuse them.

```
{webr-r}
students <- data.frame(
  name = c("John", "Jack", "Jill"),
  age = c(29, 25, 27),
  graduate = c(TRUE, TRUE, FALSE)
)

# rows and columns independently
students[2, ]
students[, "age"]

# filter by condition
students[students$age > 21, ]

# how many rows and columns
dim(students)
```

Starting with R 4.0.0 (April 2020), the default value of `stringsAsFactors` in `data.frame()` is `FALSE`, meaning character columns remain as character type. In older versions of R (< 4.0.0), the default was `TRUE`, which automatically converted character vectors to factors.

> **Common Mistake:** Assuming `data.frame()` still converts strings to factors. That was the default before R 4.0.0 and was a frequent source of confusion. On R >= 4.0.0 the default is `FALSE`.

```
students <- data.frame(name, age, graduate, stringsAsFactors = FALSE)
```

Practice Check

Knowledge Check: *One column, three operators* What do `students[1]`, `students[[1]]`, and `students$name` each return?

Click to reveal answer **Answer:** A one-column **data frame**; a **vector**; and the same **vector**.

Explanation: `students[1]`, with a single index and no comma, always returns a data frame, even for a single column, so downstream code that expects a vector breaks. `[[` and `$` both extract the column itself as a vector. Reach for `[[` or `$` whenever you want the values. **Knowledge Check:** *Rows and columns are indexed separately* What does `students[1, ]` return, and how does that differ from `students[, 1]`?

Click to reveal answer **Answer:** `students[1, ]` returns the **first row**; `students[, 1]` returns the **first column**. The comma is what separates the two.

Explanation: Inside `[` the first position selects rows and the second selects columns. Leaving one side empty means "all of them". So `students[1, ]` is row 1, `students[, 1]` is column 1, and `students[1, 1]` is the single cell where they cross. **Knowledge Check:** *Filtering rows by a condition* How do you return only the students older than 26?

Click to reveal answer **Answer:** `students[students$age > 26, ]`, which gives John (29) and Jill (27).

Explanation: The condition builds a logical vector with one value per row. Using it as the row index keeps the rows where the condition is `TRUE` and drops the rest. The column position after the comma is empty, so every column is kept. **Knowledge Check:** *How many rows and columns?* The data frame has three columns and three rows. Which two functions report those counts directly?

Click to reveal answer **Answer:** `nrow(students)` gives `3` and `ncol(students)` gives `3`. Both are also available together as `dim(students)`, which returns `c(3, 3)`.

Explanation: `dim()` returns rows first, then columns, so `dim(students)[1]` is the row count. `str()` gives you a structural overview while `nrow()` and `ncol()` answer the two specific questions directly. **Knowledge Check:** *`stringsAsFactors` changed in R 4.0.0* In R 4.0.0 and later, what does `data.frame(name, age, graduate)` do with the character column `name`?

Click to reveal answer **Answer:** It keeps it as `character`. Before R 4.0.0 the default was `TRUE`, which silently converted character columns to factors.

Explanation: The old default caused endless confusion, because strings became categories and levels were alphabetised behind your back. Passing `stringsAsFactors = FALSE` is now redundant on R >= 4.0.0, but it keeps code readable and portable to older R.

Key points

- A data frame holds equal-length columns that can have different types.
- Use `str()`, `dim()`, `nrow()` and `ncol()` to inspect its structure.
- Use `[[` or `$` to extract a column as a vector; `students[1]` keeps a data frame.
- In `students[rows, columns]`, the comma separates row and column selections.

Next: Install & Update R Packages

Practise interactively: Course 1, lesson 7

Install & Update R Packages

Learning Objectives: After this chapter you will be able to:

- Install a package from CRAN with `install.packages()`.
- Explain the difference between installing and loading a package.
- Load a package with `library()` in each R session.
- Update installed packages with `update.packages()`.

- Fix a “there is no package called” error.

Introduction

Packages add functions and data to R. Install a package once on your computer, then load it in each R session where you want to use it.

Install Packages

CRAN

We can install packages from CRAN using `install.packages()`. The name of the package must be enclosed in single or double quotes.

```
install.packages('ggplot2')
```

Load a package

`install.packages()` saves a package on your computer. `library()` makes it available in the current R session. You do not need to reinstall a package each time you open R, but you do need to load it again.

```
library(ggplot2)
```

If `library(ggplot2)` reports `there is no package called 'ggplot2'`, install it with `install.packages("ggplot2")`, then run `library(ggplot2)` again.

Update Packages

Use `update.packages()` to check for and install package updates.

```
update.packages()
```

For other package sources, archived versions and library paths, see Package management.

Key points

- Use `install.packages()` to install a package from CRAN on your computer.
- Install once, then run `library()` in each session where you need the package.
- A “there is no package called” error means you need to install that package before loading it.
- Use `update.packages()` to check for and install package updates.

Next: RStudio Projects & Data Import

RStudio Projects & Data Import

Practise this interactively in Course 1 (Introduction to R), lesson 8

Learning Objectives: After this chapter you will be able to:

- Work inside an RStudio Project (`.Rproj`) instead of `setwd()`.

- Build portable paths with `here::here()`.
- Read CSV files with `readr::read_csv()` and Excel files with `readxl::read_excel()`.

```
library(readr)
library(readxl)
library(here)
```

Why avoid `setwd()`

`setwd("C:/Users/you/project")` hard-codes a path that exists only on your machine. Anyone opening the script on a different computer gets an error. An RStudio Project (`.Rproj`) fixes this: opening the `.Rproj` file sets the working directory to the project folder automatically, so relative paths work.

Workflow: File → New Project → New Directory (or Existing Directory) → give it a name. From then on, open the `.Rproj` file, not individual scripts.

Portable paths with `here`

The `here` package builds paths from the project root, wherever the project lives on disk:

```
here("data", "students.csv")
```

```
[1] "/home/runner/work/intro-r/intro-r/data/students.csv"
```

> **Common Mistake:** Paths with `../` or `C:/...` break when the project moves. If a path starts with a drive letter or `../`, rewrite it with `here()`.

Reading CSV files

Practice data lives in the `data/` directory (CC0 / public domain, created for this book). Read it with `read_csv()`:

```
students <- read_csv(here("data", "students.csv"))
students
```

```
# A tibble: 5 x 3
  name    age score
  <chr> <dbl> <dbl>
1 Asha     21    88
2 Ravi     22    92
3 Meera    20    79
4 Arjun    23    95
5 Divya    21    84
```

Reading Excel files

`read_excel()` works the same way for `.xlsx` files. It reads sheet 1 by default:

```
students_xl <- read_excel(here("data", "students.xlsx"))
students_xl
```

```
# A tibble: 5 x 3
  name    age score
  <chr> <dbl> <dbl>
1 Asha     21    88
2 Ravi     22    92
3 Meera    20    79
4 Arjun    23    95
5 Divya    21    84
```

Try it in your browser

Edit the code below and press **Run**. It executes entirely in your browser via WebR — no R installation needed. The runtime downloads once, on this page only.

First run downloads the R runtime and any required packages (about 20 s); later runs reuse them.

```
{webr-r}
library(readr)

# the browser filesystem starts empty, so fetch the sample first
if (!file.exists("students.csv")) {
  download.file(
    "https://intro-r.rsquaredacademy.com/data-webr/students.csv",
    "students.csv"
  )
}

students <- read_csv("students.csv", show_col_types = FALSE)
students

# check the type readr chose for each column
sapply(students, class)
```

Practice Check

Knowledge Check: *Project-relative paths* Why does `read_csv(here("data", "students.csv"))` keep working when you move the whole project folder to another computer, but `read_csv("C:/Users/you/project/data/students.csv")` does not?

Click to reveal answer **Answer:** `here()` resolves the path relative to the project root at run time, so it adapts to the new location. The `C:/...` path is absolute and points to a folder that does not exist on the other computer.

Explanation: Projects + `here()` keep every path portable (`project root` → `data` → `file`); absolute paths and `setwd()` tie the script to one machine. **Knowledge Check:** *Why `.Rproj` beats `setwd()`* What breaks when you share a script containing `setwd("C:/Users/you/project")` with a colleague?

Click to reveal answer **Answer:** The script errors because that folder does not exist on your colleague's machine.

Explanation: `setwd()` with an absolute path ties the script to one computer; opening a shared `.Rproj` file sets everyone's working directory to the project folder automatically. **Knowledge Check:** *`here()` resolution* Your project lives at `D:/work/survey` and contains `data/resp.csv`. What does `here("data", "resp.csv")` return?

Click to reveal answer **Answer:** `D:/work/survey/data/resp.csv` (i.e. `project root` → `data` → `resp.csv`).

Explanation: `here()` anchors paths at the project root (where the `.Rproj` file lives), joining each additional component with the correct separator for your OS. **Knowledge Check:** *`read_csv()` vs `read.csv()`* Name one practical advantage of `readr::read_csv()` over base R's `read.csv()` for a beginner.

Click to reveal answer **Answer:** Any one of: it never converts strings to factors, it prints a compact tibble preview, or it reports column types as it reads.

Explanation: `read_csv()` uses modern defaults (no `stringsAsFactors` surprises) and tells you the parsed type of each column, so mismatches are visible immediately. **Knowledge Check:** *Excel sheets* `read_excel(here("data", "students.xlsx"))` reads which sheet by default, and how would you read the second sheet instead?

Click to reveal answer **Answer:** The first sheet by default; pass `sheet = 2` (or the sheet name) to read the second sheet.

Explanation: `read_excel()` defaults to sheet 1; the `sheet` argument selects another sheet by position or name.

Key points

- An RStudio Project keeps paths relative to a shared project folder.
- Use `here()` to build portable paths instead of hard-coding a working directory.
- Read CSV files with `read_csv()` and Excel files with `read_excel()`.

Next: Your First Visualization

Practise interactively: Course 1, lesson 8

Your First Visualization

Practise this interactively in Course 1 (Introduction to R), lesson 10

Learning Objectives: After this chapter you will be able to:

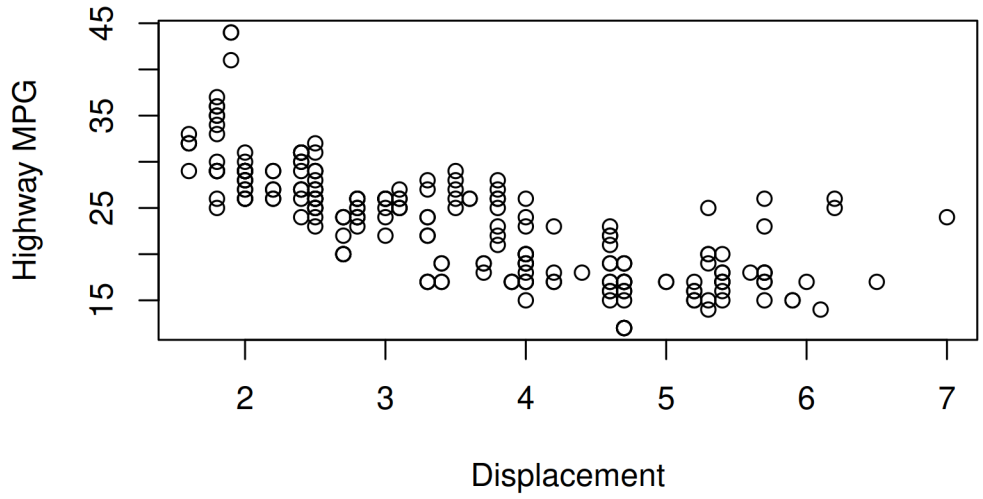
- Load `ggplot2` and its bundled `mpg` dataset.
- Draw the same scatterplot in base R and in ggplot2.
- Describe a plot as data + aesthetics + geometry.

```r
library(ggplot2)
# ggplot2 bundles the `mpg` dataset (MIT license, same as ggplot2 itself)
data(mpg, package = "ggplot2")
```

Common Mistake: Base R has no `mpg` dataset (it has `mtcars`). `plot(mpg$displ, mpg$hwy)` without `library(ggplot2)` fails with `object 'mpg' not found`. Always load `ggplot2` first.

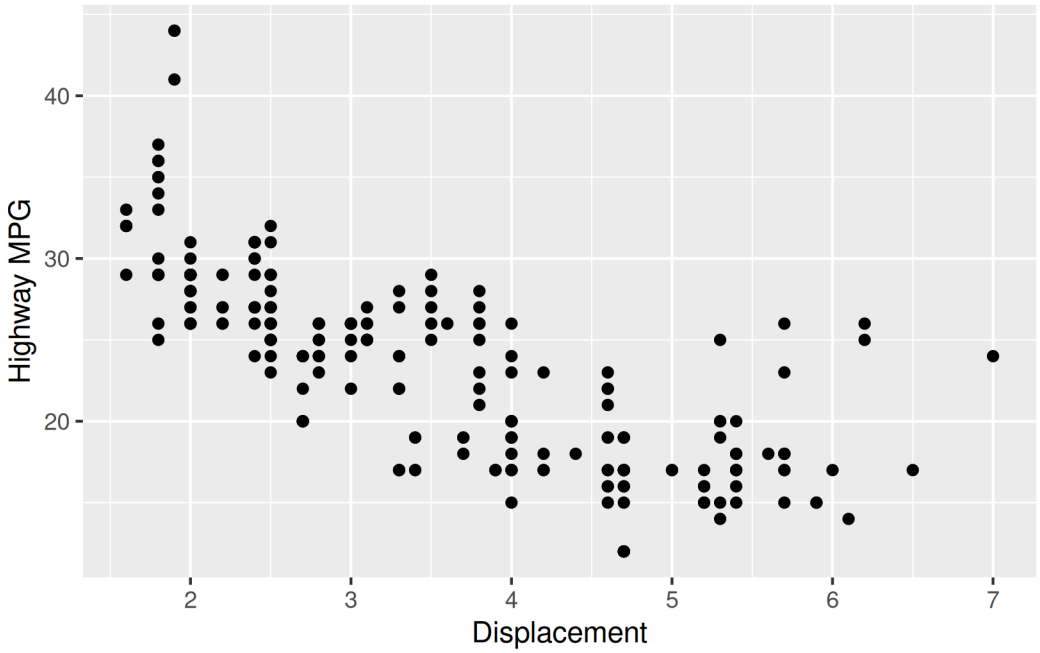
Base R graphics

```r
plot(mpg$displ, mpg$hwy, xlab = "Displacement", ylab = "Highway MPG")
```



ggplot2 graphics

```r
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  labs(x = "Displacement", y = "Highway MPG")
```



Mental model: every ggplot combines *data* (`mpg`), *aesthetics* (`aes(x, y)`) and *geometry* (`geom_point()`). Change the geometry, keep the mapping.

Try it in your browser

Edit the code below and press **Run**. It executes entirely in your browser via WebR — no R installation needed. The runtime downloads once, on this page only.

First run downloads the R runtime and any required packages (about 20 s); later runs reuse them.

```
{webr-r}
library(ggplot2)

p <- ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(color = "steelblue") +
  labs(x = "Displacement", y = "Highway MPG")
p

# same data and mapping, different geometry
p + geom_smooth(method = "lm", se = FALSE)
```

Practice Check

Knowledge Check: *Data + aesthetics + geometry* In `ggplot(mpg, aes(x = displ, y = hwy)) + geom_point()`, which part is the data, which is the aesthetics mapping, and which is the geometry?

Click to reveal answer **Answer:** Data: `mpg`. Aesthetics: `aes(x = displ, y = hwy)`. Geometry: `geom_point()`.

Explanation: `ggplot()` takes the data and the variable mapping; each `geom_*()` layer decides how those mapped variables are drawn. **Knowledge Check:** *Why `library(ggplot2)` comes first* What error do you get from `plot(mpg$displ, mpg$hwy)` in a fresh R session, and why?

Click to reveal answer **Answer:** `object 'mpg' not found` — base R has no `mpg` dataset (it has `mtcars`); `mpg` only exists after `library(ggplot2)` (or `data(mpg, package = "ggplot2")`).

Explanation: Datasets live inside packages. Until the package is loaded, its data is invisible to your session. **Knowledge Check:** *Changing the geometry* How would you turn the scatterplot into a smoothed trend line plot without changing the data or the variable mapping?

Click to reveal answer **Answer:** Replace `geom_point()` with `geom_smooth()` (or add it as a second layer).

Explanation: Data + aesthetics stay fixed while geometries are interchangeable layers — that is the point of the grammar of graphics. **Knowledge Check:** *Aesthetics mapping* In `aes(x = displ, y = hwy)`, what do `x` and `y` refer to — plot labels or dataset columns?

Click to reveal answer **Answer:** Dataset columns: `mpg$displ` mapped to horizontal position, `mpg$hwy` to vertical position.

Explanation: `aes()` maps columns of the data to visual properties. Labels are set separately with `labs()`. **Knowledge Check:** *Base R vs ggplot2* The base R call repeats `mpg$` twice; the ggplot2 call mentions `mpg` once. Why is that difference useful as plots get more complex?

Click to reveal answer **Answer:** Declaring the data once in `ggplot(mpg, ...)` means every layer inherits it — you never retype the data source per variable, so longer plots stay readable and less error-prone.

Explanation: One data declaration shared across layers beats repeating `data$column` for every variable, especially with facets and multiple geoms.

Key points

- Load ggplot2 before using its bundled `mpg` dataset.
- Base R and ggplot2 can draw the same scatterplot.
- A ggplot combines data, an aesthetics mapping and a geometry.

Next: Your First Data Wrangling

Practise interactively: Course 1, lesson 10

Your First Data Wrangling

Learning Objectives: After this chapter you will be able to:

- Use the five core `dplyr` verbs: `filter()`, `select()`, `mutate()`, `arrange()`, `summarise()`.
- Chain steps with the native base pipe `|>`.
- Know where to go next after this book.

```r
library(dplyr)
# ggplot2 bundles the `mpg` dataset (MIT license, same as ggplot2 itself)
data(mpg, package = "ggplot2")
```

We use the native base pipe `|>` (standard since R 4.1.0; no external dependencies). We do **not** use `%>%`.

The five verbs

```r
mpg |>
  filter(manufacturer == "audi") |>
  select(manufacturer, model, displ, hwy) |>
  mutate(km_per_litre = hwy * 0.425144) |>
  arrange(desc(hwy))
```

```
# A tibble: 18 x 5
   manufacturer model      displ   hwy km_per_litre
   <chr>        <chr>      <dbl> <int>        <dbl>
 1 audi         a4           2      31        13.2
 2 audi         a4           2      30        12.8
 3 audi         a4           1.8    29        12.3
 4 audi         a4           1.8    29        12.3
 5 audi         a4 quattro   2      28        11.9
 6 audi         a4           3.1    27        11.5
 7 audi         a4 quattro   2      27        11.5
 8 audi         a4           2.8    26        11.1
 9 audi         a4           2.8    26        11.1
10 audi         a4 quattro   1.8    26        11.1
11 audi         a4 quattro   1.8    25        10.6
12 audi         a4 quattro   2.8    25        10.6
13 audi         a4 quattro   2.8    25        10.6
14 audi         a4 quattro   3.1    25        10.6
15 audi         a4 quattro   3.1    25        10.6
16 audi         a6 quattro   3.1    25        10.6
17 audi         a6 quattro   2.8    24        10.2
18 audi         a6 quattro   4.2    23         9.78
```

```r
mpg |>
  summarise(mean_hwy = mean(hwy), n = n())
```

```
# A tibble: 1 x 2
  mean_hwy     n
     <dbl> <int>
1     23.4   234
```

Try it in your browser

Edit the code below and press **Run**. It executes entirely in your browser via WebR — no R installation needed. The runtime downloads once, on this page only.

First run downloads the R runtime and any required packages (about 20 s); later runs reuse them.

```
{webr-r}
library(dplyr)

# filter, select, mutate, arrange
mpg |>
  filter(manufacturer == "audi") |>
  select(model, hwy) |>
  mutate(km_per_litre = hwy * 0.425144) |>
  arrange(desc(hwy)) |>
  head(5)

# summarise collapses to a single row
mpg |>
  summarise(mean_hwy = mean(hwy), n = n())
```

Practice Check

Knowledge Check: *Filter then select* What does `mpg |> filter(manufacturer == "audi") |> select(model, hwy)` return?

Click to reveal answer **Answer:** A data frame with two columns (`model`, `hwy`) containing only the rows where `manufacturer` is `"audi"`.

Explanation: `filter()` keeps matching rows; `select()` keeps matching columns. The `|>` pipe passes the left-hand result into the first argument of the right-hand function. **Knowledge Check:** *`mutate()` adds columns* What does `mpg |> mutate(km_per_litre = hwy * 0.425144)` return, and how many rows does the result have compared to `mpg`?

Click to reveal answer **Answer:** The full `mpg` data with one extra column (`km_per_litre`); the row count is unchanged.

Explanation: `mutate()` adds (or modifies) columns row-wise — it never adds or removes rows. Use `filter()` to change row counts. **Knowledge Check:** *`arrange()` ordering* After `mpg |> arrange(desc(hwy))`, which kind of car sits in row 1?

Click to reveal answer **Answer:** The car with the highest highway mileage (`hwy`).

Explanation: `arrange()` sorts ascending by default; wrapping in `desc()` flips it to descending, so the maximum comes first. **Knowledge Check:** *`summarise()` collapses rows* How many rows does `mpg |> summarise(mean_hwy = mean(hwy), n = n())` return, and why?

Click to reveal answer **Answer:** Exactly one row: the mean of `hwy` across the whole dataset and the total row count.

Explanation: Without `group_by()`, `summarise()` collapses the entire data frame into a single summary row. **Knowledge Check:** *Reading a pipeline* Read this pipeline aloud in order: `mpg |> filter(manufacturer == "audi") |> select(model, hwy) |> arrange(desc(hwy))`. What question does it answer?

Click to reveal answer **Answer:** "Among Audi models, which has the best (and worst) highway mileage?" — it keeps only Audis, keeps the model and mileage columns, and sorts best-first.

Explanation: Pipelines read top-to-bottom as a sequence of verbs; narrating each step in plain words is the fastest way to check you built the right one.

Where to go next

> **Where to go next:** - Book: R for Data Science (2e), starting at Chapter 1 - Courses: Rsquared Academy Data Wrangling series - More books: ebooks.rsquaredacademy.com

Key points

- Use `filter()` for rows and `select()` for columns.
- Use `mutate()` to add or change columns and `arrange()` to sort rows.
- Use `summarise()` for summaries and `n()` for the row count.
- The native pipe `|>` passes each result to the next step.

Next: Factors in R

Practise interactively: Course 1, lesson 9

Factors in R

> **Learning Objectives:** After this chapter you will be able to:

- Create a factor from a character vector.
- Inspect and count levels with `levels()` and `nlevels()`.
- Impose a natural order with `levels=` together with `ordered = TRUE`.
- Attach shorter display labels with `labels=`.
- Test for ordering with `is.ordered()`.
- Reach for `forcats` when you need to recode, reorder, or lump levels.

Introduction

In R, we represent categorical or qualitative data with the data type `factor`.

Create Factors

```
args(factor)
```

```
function (x = character(), levels, labels = levels, exclude = NA,
    ordered = is.ordered(x), nmax = NA)
NULL
```

```
devices <- factor(c('Mobile', 'Tablet', 'Desktop'))
devices
```

```
[1] Mobile  Tablet  Desktop
Levels: Desktop Mobile Tablet
```

```
# number of levels
nlevels(devices)
```

```
[1] 3
```

```
# levels
levels(devices)
```

```
[1] "Desktop" "Mobile"  "Tablet"
```

Order Levels

Use the `levels` argument together with `ordered = TRUE` when the categories have a natural order:

```
size <- factor(c('Medium', 'Low', 'High', 'Low', 'Medium'),
  levels = c('Low', 'Medium', 'High'), ordered = TRUE)
size
```

```
[1] Medium Low    High   Low    Medium
Levels: Low < Medium < High
```

```
levels(size)
```

```
[1] "Low"    "Medium" "High"
```

Specify Labels

Use the `labels` argument to attach shorter display labels while keeping the underlying level order:

```
size_labeled <- factor(c('Low', 'Medium', 'High'),
  levels = c('Low', 'Medium', 'High'), labels = c('L', 'M', 'H'))
size_labeled
```

```
[1] L M H
Levels: L M H
```

Check Levels

```
# existing levels
levels(size)
```

```
[1] "Low"    "Medium" "High"
```

```
# number of levels
nlevels(size)
```

```
[1] 3
```

```
# is this an ordered factor?
is.ordered(size)
```

```
[1] TRUE
```

To reorder, recode or lump rare levels, see the forcats package.

Key points

- Factors represent categorical data with a set of levels.
- Use `levels()` and `nlevels()` to inspect the categories.
- Set `levels` and `ordered = TRUE` when categories have a natural order.
- Use `labels` to choose display labels and `is.ordered()` to check ordering.

Next: Lists in R

Lists in R

Learning Objectives: After this chapter you will be able to:

- Create a list that holds elements of different types.
- Name list elements with `names()`.
- Access elements with `[[`, `[`, and `$`.
- Explain why `[[` returns the element while `[` returns a list.
- Convert a vector or matrix into a list with `as.list()`.
- Convert a list back into a vector with `unlist()`.

Introduction

Lists are very useful as they are heterogeneous: they can contain different data types. Vectors and matrices are homogeneous: they can contain only one type of data. A matrix is a rectangular arrangement of values of one type, with rows and columns. If you include different data types in a vector or matrix, R coerces them to the same type. With lists, it is different. In this chapter, we will learn more about lists.

Creating Lists

To create a list, we use the `list()` function. Let us create a simple list to demonstrate how it can contain different data types.

```
# numeric vector
vect1 <- 1:10

# character vector
vect2 <- c('Jack', 'John', 'Jill')

# logical vector
vect3 <- c(TRUE, FALSE)

# matrix
mat <- matrix(data = 1:9, nrow = 3)

# list
l <- list(vect1, vect2, vect3, mat)
l
```

```
[[1]]
 [1]  1  2  3  4  5  6  7  8  9 10

[[2]]
[1] "Jack" "John" "Jill"

[[3]]
[1]  TRUE FALSE

[[4]]
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

In the output, all the elements of the list retain their data types. Let us learn how to access the elements of the list.

Access List Elements

You can access the elements of a list using the following operators:

- `[[`
- `[`
- `$`

Let us try them one by one.

```
# using [[
l[[1]]
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```
# using [
l[1]
```

```
[[1]]
 [1]  1  2  3  4  5  6  7  8  9 10
```

`[[` extracts the element itself (which may be a vector, matrix, data frame, or nested list), while `[` returns a new list containing the selected element(s). We can use the `$` operator only when we have named elements in the list. Let us add names to the elements using the `names()` function.

```
# named elements
names(l) <- c('vect1', 'vect2', 'vect3', 'mat')
l
```

```
$vect1
 [1]  1  2  3  4  5  6  7  8  9 10

$vect2
[1] "Jack" "John" "Jill"
```

```
$vect3
[1]  TRUE FALSE

$mat
     [,1] [,2] [,3]
[1,]    1    4    7
[2,]    2    5    8
[3,]    3    6    9
```

```
# use $
l$vect1
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```
# use [[
l[['vect1']]
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

```
# use [
l['vect1']
```

```
$vect1
 [1]  1  2  3  4  5  6  7  8  9 10
```

Coerce other objects

You can coerce other objects to a list using `as.list()`.

```
# vector
vect1 <- 1:10
as.list(vect1)
```

```
[[1]]
[1] 1

[[2]]
[1] 2

[[3]]
[1] 3

[[4]]
[1] 4

[[5]]
[1] 5

[[6]]
[1] 6

[[7]]
[1] 7
```

```
[[8]]
[1] 8

[[9]]
[1] 9

[[10]]
[1] 10
```

```
# matrix
mat <- matrix(data = 1:9, nrow = 3)
as.list(mat)
```

```
[[1]]
[1] 1

[[2]]
[1] 2

[[3]]
[1] 3

[[4]]
[1] 4

[[5]]
[1] 5

[[6]]
[1] 6

[[7]]
[1] 7

[[8]]
[1] 8

[[9]]
[1] 9
```

Coerce list to other objects

Use `unlist()` to coerce a list to a vector.

```
# unlist
unlist(l)
```

```
 vect11  vect12  vect13  vect14  vect15  vect16  vect17  vect18  vect19 vect110
    "1"     "2"     "3"     "4"     "5"     "6"     "7"     "8"     "9"    "10"
 vect21  vect22  vect23  vect31  vect32    mat1    mat2    mat3    mat4    mat5
 "Jack"  "John"  "Jill"  "TRUE" "FALSE"     "1"     "2"     "3"     "4"     "5"
```

```
   mat6    mat7    mat8    mat9
    "6"     "7"     "8"     "9"
```

```
# remove names
unlist(l, use.names = FALSE)
```

```
 [1] "1"     "2"     "3"     "4"     "5"     "6"     "7"     "8"     "9"
[10] "10"    "Jack"  "John"  "Jill"  "TRUE"  "FALSE" "1"     "2"     "3"
[19] "4"     "5"     "6"     "7"     "8"     "9"
```

Practice Check

Knowledge Check: *Why use a list at all?* `vect1 <- 1:10`, `vect2 <- c('Jack', 'John', 'Jill')`, and `vect3 <- c(TRUE, FALSE)`. Why not just `c(vect1, vect2, vect3)`?

Click to reveal answer **Answer:** That would coerce everything to `character`, because `character` is the most flexible type. A list keeps each element's own type.

Explanation: `c(vect1, vect2, vect3)` gives you `c("1", "2", ..., "10", "Jack", ...)`. Vectors and matrices are homogeneous, so mixing types destroys information. `list(vect1, vect2, vect3)` stores the three vectors side by side with their types intact. **Knowledge Check:** *`[[` versus `[`* Given `l <- list(1:10, c('Jack', 'John'))`, what is the difference between `l[[1]]` and `l[1]`?

Click to reveal answer **Answer:** `l[[1]]` is the vector `1:10` itself. `l[1]` is a list containing one element, which is that same vector.

Explanation: `[[` extracts the element, so you get the object and can operate on it directly. `[` builds a new list holding the selection, which is what you want when you want to delete or keep several elements. Square brackets are for selecting; double square brackets are for extracting. **Knowledge Check:** *`$` needs names* Why does `l$vect1` fail before you run `names(l) <- c('vect1', 'vect2', 'vect3', 'mat')`?

Click to reveal answer **Answer:** The elements have no names, so there is nothing for `$` to match.

Explanation: `$` looks up elements **by name**, not by position. Until the list has names, use `l[[1]]` or `l[1]`. Once named, `l$vect1` and `l[['vect1']]` both return the vector. **Knowledge Check:** *Coercing to a list* How do you turn the vector `vect1 <- 1:10` into a list, one element per value?

Click to reveal answer **Answer:** `as.list(vect1)`, which gives a list of ten single numbers.

Explanation: `as.list()` splits the vector into its individual elements. The same call works on a matrix, in which case every value of the matrix becomes its own element (nine elements for a 3 x 3 matrix, taken column by column). The reverse direction is `unlist()`. **Knowledge Check:** *Flattening a list* What does `unlist(l, use.names = FALSE)` do differently from `unlist(l)`?

Click to reveal answer **Answer:** It returns the same values but without the names.

Explanation: `unlist()` concatenates every element into one vector and keeps the names it finds, so you get a named vector. Setting `use.names = FALSE` drops the names, which is usually what you want for further calculation.

Key points

- A list can hold elements of different types without coercing them to one type.
- Use `[[` to extract an element and `[` to keep a selection as a list.
- Name list elements to access them with `$`.
- Use `as.list()` to create a list and `unlist()` to flatten one into a vector.

Next: Getting Help in R

Getting Help in R

Learning Objectives: After this chapter you will be able to:

- Look up any known function with `?name` or `help()`.
- Search help topics when you do not know the function name.
- Read a package's documentation, examples, and vignettes.
- Decide when the built-in help system is enough and when to go online.

Introduction

In this chapter, we will learn about the different methods of getting help in R. We often get stuck during an analysis because we do not know which function to use or its syntax. Anyone new to R should know where to look for help. There are two ways to look for help in R:

- online
- built-in help system

In the first section, we will look at online resources. We will then look at the built-in help system and package documentation. Let us get started!

Online Resources

R Bloggers

R Bloggers aggregates blogs written in English from across the globe. This is the first place to visit for help with R, data analysis, visualization and machine learning. There are blogs on a wide range of topics and the latest content is delivered to your inbox if you subscribe. If you are a blogger yourself, share it with the R community by adding your blog to R Bloggers.

Stack Overflow

Stack Overflow is a place to ask for help with R code or packages. Someone may have encountered a similar problem, and you can use the answers given by R experts. If you encounter a new problem, you can ask for help by providing a reproducible example of your analysis along with the R code. Use the reprex package to create reproducible examples.

Posit Community

Posit Community is similar to Stack Overflow. You can ask questions related to RStudio, Shiny and tidyverse. You can also use #rstats on Mastodon or Bluesky.

RStudio

RStudio has resources including cheatsheets, webinars and blogs.

Reddit

Reddit is another place to look for help. The discussions are moderated by R experts. There are subreddits for Rstats, Rlanguage, Rstudio and Rshiny.

R Weekly

Visit RWeekly to get regular updates about the R community. You can find information about new packages, blogs, conferences, workshops, tutorials and R jobs.

R User Groups

Several R User Groups are active across the globe. You can find the list here. Join the local user group to meet, discuss and learn from other R enthusiasts and experts.

Internal

We can use the following functions to access the built-in help system:

- `help.start()`
- `help()`
- `?`
- `??`
- `help.search()`
- `demo()`
- `example()`
- `library()`
- `vignette()`
- `browseVignettes()`

help.start

The `help.start()` function opens the documentation page in your browser. Here you can find manuals, references and other materials.

```
help.start()
```

help

Use `help()` to access the documentation of functions and data sets. `?` is a shortcut for `help()` and returns the same information.

```
help(plot)
?plot
```

help.search

`help.search()` searches all sources of documentation and returns those that match the search string. `??` is a shortcut for `help.search()` and returns the same information.

```
help.search('regression')
??regression
```

demo

demo displays an interactive demonstration of certain topics provided in an R package. Typing `demo()` in the console lists the demos available in installed R packages.

```
demo()
demo(scoping)
```

example

`example()` displays examples of the specified topic if available.

```
example('mean')
```

```
mean> x <- c(0:10, 50)

mean> xm <- mean(x)

mean> c(xm, mean(x, trim = 0.10))
[1] 8.75 5.50
```

Package Documentation

library

Access the documentation of a package using the `help` argument to `library()`. The package must be installed. For packages you have not installed, read the CRAN page or its pkgdown site.

```
library(help = 'ggplot2')
```

vignette

A vignette is a long-form guide to an R package. You can access vignettes using `vignette()`. It displays a list of vignettes available in installed packages.

```
vignette()
```

To access a specific vignette from a package, specify the name of the vignette and the package.

```
vignette('dplyr', package = 'dplyr')
```

browseVignettes

`browseVignettes()` is another way to access the vignettes in installed packages. It lists the vignettes in each package with links to the web page and R code.

```
browseVignettes()
```

RSiteSearch

`RSiteSearch()` searches for specified topics in help pages, vignettes and task views using the search engine at this link. It returns the result in a browser.

```
RSiteSearch('glm')
```

Key points

- Use `help()` or `?` when you know the function name.
- Use `help.search()` or `??` to search documentation by topic.
- `library(help = ...)` and `vignette()` show documentation for installed packages.
- When asking online, share your R code and a reproducible example.

Next: About the Author

About the Author

Aravind Hebbali is the founder of Rsquared Academy. He earned his Masters in Economics from Madras School of Economics. As an active R user, he has authored several R packages such as

- olsrr
- rfm
- descriptr
- blorr
- xplorerr

In 2015, he founded Rsquared Academy, a free and open source education initiative with a focus on data science and analytics. Apart from self paced online courses, Rsquared Academy offers customized learning modules for corporates and universities.

You can find him on GitHub.

Package management

Reference material for managing packages, followed by background on R and its installation tools. For the basics, start with Install & Update R Packages.

Introduction

This appendix covers package management beyond the basics. Packages are fundamental to R. More than 22,000 packages are available on CRAN, the Comprehensive R Archive Network (as of September 2026; verify the current count at cran.r-project.org/web/packages).

Packages are available for different topics. You should always look for a package before writing code from scratch. If you have written your own code for a new analysis or topic, do share it with the R community by converting it into a package. You can learn more about building R packages from R Packages, a book by Hadley Wickham.

Other package sources

GitHub

Some developers make R packages available on GitHub before releasing them on CRAN. We can install such packages using `install_github()` from the devtools or remotes package. You need

to specify the name of the repository and the package. For example, we can download ggplot2 or dplyr with this code:

```
devtools::install_github("tidyverse/ggplot2")
remotes::install_github("tidyverse/dplyr")
```

Bioconductor

Bioconductor provides tools for analysis and comprehension of high throughput genomic data. Install the `BiocManager` package and use `BiocManager::install()`:

```
# install BiocManager if needed
install.packages("BiocManager")

# install a Bioconductor package
BiocManager::install("GenomicFeatures")
```

> For faster installs with dependency resolution, you can also try `pak::pkg_install()` from the pak package as a modern alternative.

Install Different Versions

Now that we have learned how to install packages, let us look at installing different versions of the same package.

```
remotes::install_version('dplyr', version = "0.5.0")
```

If you want to install the latest release from GitHub, append `@*release` to the repository name. For example, to install the latest release of dplyr:

```
remotes::install_github('tidyverse/dplyr@*release')
```

Installed Packages

- `installed.packages()`: view currently installed packages
- `autoload('function_name', 'package_name')`: load functions and data from packages only when called in the script
- `available.packages()`: packages available for installation
- `old.packages()`: packages which have new versions available
- `new.packages()`: packages not yet installed
- `remove.packages('package_name')`: remove installed packages

Library Paths

A library is a directory that contains installed packages. Your system will usually have more than one R library. You can locate the libraries using `.libPaths()`.

```
.libPaths()
```

```
[1] "/home/runner/.cache/R/renv/library/intro-r-eca02dad/linux-ubuntu-noble/
R-4.5/x86_64-pc-linux-gnu"
```

```
[2] "/home/runner/.cache/R/renv/sandbox/linux-ubuntu-noble/R-4.5/x86_64-pc-linux-
gnu/46e831e3"
```

Use `lib` to install or remove packages in a particular library, and `lib.loc` to load or update them.

Install

```
install.packages('stringr', lib = .libPaths()[1])
```

Load

```
library(lubridate, lib.loc = .libPaths()[1])
```

Update Packages

```
update.packages(lib.loc = .libPaths()[1])
```

Remove Packages

```
remove.packages('stringr', lib = .libPaths()[1])
```

Explore Rsquared Academy's own R packages and guides at pkgs.rsquaredacademy.com.

R background and installation tools

The rest of this appendix gives background on the organizations behind R and the tools used to install it.

Organizations

R Foundation

The R Foundation is a not-for-profit organization founded by members of the R core team. It is responsible for the continued development of R. It also supports the annual useR conferences. You can learn more about the R Foundation here.

R Consortium

The R Consortium supports the R Foundation and other key organizations and groups developing, maintaining and using R. It funds projects that improve the infrastructure around R and supports conferences and user groups around the world. You can learn more about the R Consortium here.

Installation

R is available for Windows, Mac and is part of many Linux distributions. You can download and install R from any of the CRAN mirrors listed here.

CRAN

CRAN stands for **C**omprehensive **R** **A**rchive **N**etwork. You can download R and user contributed packages from here for different operating systems. CRAN is pronounced as KRAN or C-RAN.

Extensions

User contributed packages can be downloaded from

- CRAN

- Bioconductor
- GitHub
- R-Forge

R GUI

R comes with a simple GUI with sufficient features for data analysis and visualization.

